CryptoChain Analyzer Dashboard

Final Project Report

Cryptography - Hash Functions and Blockchain

Student	Jorge Sanz Romera
Project Type	Individual Project
Language	English
Main API	Mempool.space
Framework	Streamlit
Repository	blockchain-dashboard-jsanzrom

This report documents the design, implementation, and evaluation of a real-time Bitcoin dashboard developed for the CryptoChain Analyzer assignment. The project combines applied cryptography, blockchain data engineering, and AI-based analytics in a single interactive interface.

1. Executive Summary

The goal of the project was to build a professional Python dashboard capable of connecting to a public Bitcoin API, extracting live blockchain data, and transforming that data into meaningful cryptographic and analytical views. The resulting application was implemented with Streamlit and organized as a modular dashboard with seven components: four core modules aligned with the assignment requirements and three optional extensions designed to increase technical depth and presentation quality.

From a cryptographic perspective, the strongest contribution of the project is the local verification of Bitcoin block headers. The application retrieves the latest block, reconstructs the 80-byte header, applies double SHA-256 with Python's hashlib, converts the compact bits field into a full target, and checks whether the calculated hash remains below that target. This workflow directly connects the theory of Proof of Work with live network data.

From a data and visualization perspective, the dashboard provides a practical overview of the state of the Bitcoin network. It exposes live block metrics, recent timing behaviour, difficulty adjustment history, Merkle proof verification, and a security-oriented module that estimates the cost of a 51% attack. In addition, it includes two analytical modules focused on anomaly detection and comparative AI-driven classification of block inter-arrival times.

The final project therefore demonstrates not only that the application runs, but also that the student understands the cryptographic logic behind the values displayed, can organize a software project with progressive commits, and can transform raw blockchain data into a polished and explainable analytical tool.

2. System Architecture and Development Approach

The project was organized around a lightweight but clean architecture. A central API client handles all communication with Mempool.space, while each dashboard tab is implemented in a separate module. This structure makes the code easier to maintain, simplifies debugging, and allows each feature to be implemented and tested in isolation before being integrated into the final Streamlit application.

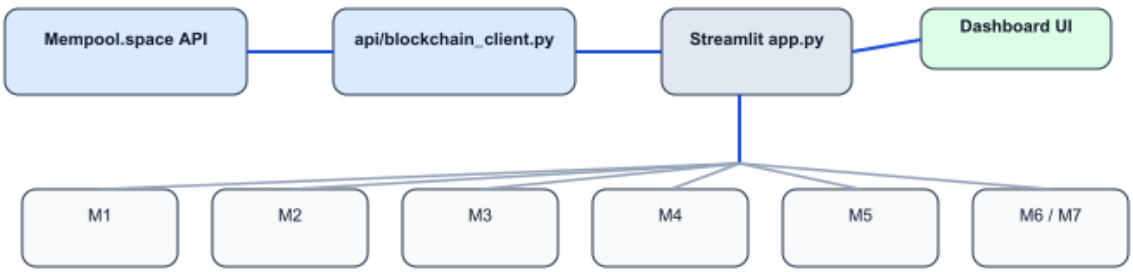

Figure 1. High-level structure of the CryptoChain Analyzer Dashboard.

A key development decision was to use Mempool.space as the main data source. This API provides direct access to live block data, block heights, transaction identifiers, and Merkle proofs, which made it especially suitable for modules focused on Proof of Work, block header parsing, and Merkle tree validation. Streamlit was selected as the dashboard framework because it supports fast iteration, integrated widgets, and clear visual organization while remaining fully in Python.

The project was developed incrementally. Instead of trying to build all features at once, each module was implemented as a separate phase and committed independently to GitHub. This workflow supported the academic requirement of progressive work and also reduced technical risk, because each stage could be tested before moving to the next one.

3. Module M1 - Proof of Work Monitor

The first module acts as the operational entry point of the dashboard. Its purpose is to monitor the latest Bitcoin block and display the most relevant values involved in Proof of Work. The module presents the current block height, number of transactions, nonce, bits, timestamp, block hash, and difficulty. These values are not hardcoded; they are retrieved live from the API every time the module is used.

Beyond the latest block, M1 also computes recent inter-arrival times between blocks. To do that, the application retrieves a configurable number of recent blocks, sorts them by height, computes the timestamp difference between consecutive blocks, and visualizes the result in a bar chart. This provides an intuitive operational view of how real block intervals fluctuate around Bitcoin's theoretical target of 600 seconds.

- Live monitoring of current block state.
- Immediate connection between bits, target, and block hash.
- Timing analysis across recent blocks.
- Clear operational presentation for a non-static dashboard.

This module is important because it gives context to the rest of the project. It introduces the user to the Bitcoin block structure, highlights the dynamic nature of mining, and establishes the visual language of the dashboard.

4. Module M2 - Block Header Analyzer

The second module is the most directly linked to cryptographic correctness. It focuses on the six fundamental fields of the Bitcoin block header: version, previous block hash, Merkle root, timestamp, bits, and nonce. Using these values from the latest block, the application reconstructs the exact 80-byte block header in the correct byte order.

Once the header has been serialized, the system computes SHA256(SHA256(header)) locally. The resulting digest is converted back into the conventional big-endian representation and then compared against the hash returned by the API. In addition, the compact bits field is decoded into the full target integer. Finally, the module verifies whether the calculated hash is numerically smaller than the target, which is the essential condition of Bitcoin Proof of Work.

Verification step	Implemented action
Header reconstruction	Serialize version, prev hash, Merkle root, timestamp, bits, nonce into 80 bytes
Hash calculation	Apply double SHA-256 with hashlib
Target decoding	Convert compact bits field into full target integer
Validation	Check both hash equality with API value and hash < target
Extra indicator	Count leading zero bits in the final block hash

This module is the strongest evidence that the dashboard is not simply a visualization layer over blockchain data. It proves that the project performs genuine cryptographic processing locally and that the values presented in the interface are tied to the theory of Bitcoin mining rather than copied blindly from an external API.

5. Module M3 - Difficulty History

Module M3 studies Bitcoin difficulty across recent completed adjustment periods. Since the Bitcoin protocol updates mining difficulty every 2016 blocks, the implementation groups data by full adjustment intervals and retrieves the start and end blocks of each period. For each completed interval, the application computes the average block time and the ratio between the actual average and the theoretical 600-second target.

The module therefore does not only display a difficulty trend. It also explains why that trend exists. If blocks arrive faster than 600 seconds on average, the ratio falls below one and the following adjustment tends to increase the difficulty. If blocks arrive more slowly, the ratio exceeds one and the system becomes relatively less demanding in the next adjustment.

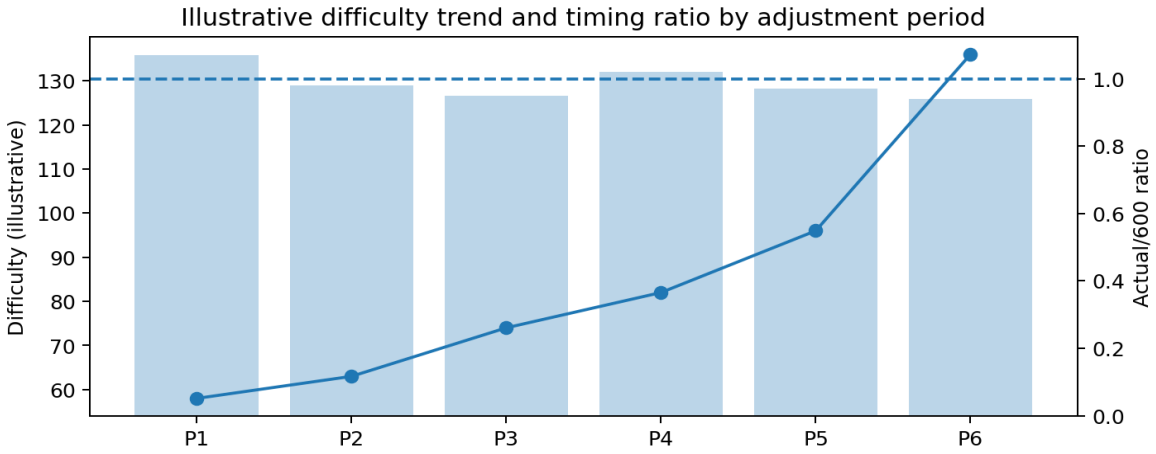

Figure 2. Illustrative representation of difficulty history and timing ratio by adjustment period.

The final dashboard version presents this module with summary metrics, a difficulty line chart, a second chart focused on the actual/600 ratio, and a table that consolidates the key outputs for each adjustment event. This makes the module useful both academically and visually: the chart communicates the network trend, while the table helps interpret individual periods.

6. Module M4 - AI Component: Z-score Anomaly Detection

The first AI-oriented module applies a simple but interpretable anomaly detection method to block inter-arrival times. After retrieving a configurable number of recent blocks, the dashboard computes the interval between consecutive timestamps and derives the mean and standard deviation of those intervals. It then calculates the z-score of each block interval and flags values with large absolute scores as anomalies.

The practical motivation is straightforward. Bitcoin blocks are expected to arrive stochastically around a target of 600 seconds, but unusually short or unusually long intervals can still be interesting for analysis. They may reflect random natural variation, periods of network instability, or unusual behaviour in mining activity. A z-score baseline is useful because it is fast to compute, transparent to explain, and easy to compare with a second AI approach.

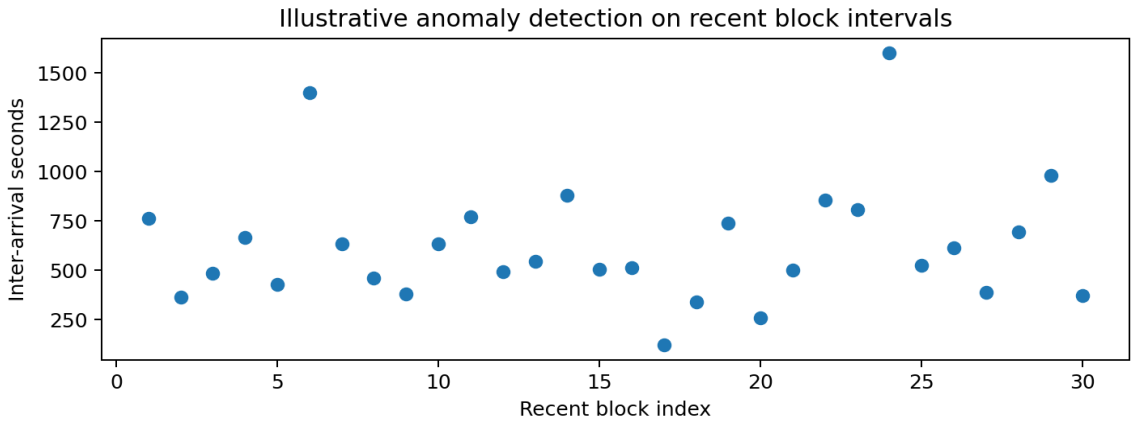

Figure 3. Illustrative anomaly-detection view for recent block intervals.

In the dashboard, this module reports the average block time, standard deviation, number of detected anomalies, and a split between fast and slow anomalies. It also plots a scatter chart of block intervals and isolates the anomalous points in a dedicated result table. While this is not a supervised machine-learning model, it remains a legitimate analytical component because it extracts structure from real data and turns it into explainable classifications.

7. Module M5 - Merkle Proof Verifier

Module M5 extends the project from block-level verification to transaction-level inclusion proofs. It selects a transaction from the latest block, requests its Merkle proof from the API, and then recomputes the path from that transaction hash to the block's Merkle root. At each level, the current hash is concatenated with its sibling in the correct order, double-hashed, and promoted to the next tree level until the root is reached.

This module is particularly valuable because it connects the theory of Merkle trees with a live, concrete example. Rather than describing inclusion proofs abstractly, the dashboard shows them as a deterministic chain of hash operations. The final result compares the recomputed root with the block's actual Merkle root and explicitly states whether the proof is valid.

In addition to the final validity result, the dashboard displays a step-by-step computation table. That table makes the module useful for both demonstration and review: the user can see the sibling hashes, the direction of concatenation, and the resulting parent hash at each level. This is an excellent example of turning a cryptographic concept into an inspectable analytical process.

8. Module M6 - Security Score

The M6 extension focuses on security economics. It estimates the current Bitcoin network hash rate using the common approximation hashrate = difficulty * 2^32 / 600, derives the approximate hash rate needed to control 51% of the network, and then translates that value into an estimated hourly attack cost using a configurable market rental price per TH/s.

This module is useful because it moves beyond protocol mechanics and into practical attack feasibility. Instead of describing a 51% attack only in qualitative terms, it provides a numeric estimate that links mining difficulty to economic resistance. It also includes a confirmation-based security visualization that illustrates a simple principle: the deeper a transaction is buried under subsequent confirmations, the less realistic a successful attack becomes.

Even though the confirmation-risk plot is an illustrative model rather than a full derivation of Nakamoto's exact formula, it serves the dashboard well by giving the user an intuitive security narrative. Combined with a sensitivity chart over several rental-price assumptions, M6 adds both research value and presentation strength to the final project.

9. Module M7 - Second AI Approach

The final extension introduces a second analytical method so that the dashboard does not rely on a single anomaly-detection style. In this module, recent block inter-arrival times are classified through empirical quantile thresholds instead of z-scores. Blocks below a lower quantile are labelled as unusually fast, blocks above an upper quantile are labelled as unusually slow, and the rest are classified as normal.

This approach is useful for two reasons. First, it allows direct comparison with M4, which is based on standard deviation and assumes a mean-centred interpretation of the data. Second, it is robust as a descriptive baseline because it does not require the distribution to look symmetric. The dashboard therefore gains a comparative AI section rather than a single one-off anomaly detector.

By keeping both M4 and M7 visible in the interface, the final project demonstrates methodological awareness. The same underlying data can be interpreted through different analytical lenses, and the dashboard makes that comparison explicit.

10. Evaluation of the Final Project

From a grading perspective, the completed project addresses all the core evaluation areas of the assignment. Cryptographic correctness is supported by the local block-header verification in M2 and the transaction-level Merkle proof validation in M5. Functionality is demonstrated by the fact that the dashboard consumes live blockchain data through a public API and exposes multiple interactive modules. Visualization quality is strengthened by the use of summary metrics, plots, tables, and explanatory callouts across the interface. The AI component is covered not by a superficial placeholder, but by two distinct analytical approaches in M4 and M7.

Criterion	How the project addresses it
Cryptographic correctness	M2 verifies local PoW; M5 validates Merkle inclusion proofs
AI component	M4 applies z-score anomaly detection; M7 adds quantile-based comparative analysis
Functionality and real-time data	Live retrieval of latest block, block history, txids, and proofs through API client
Visualization	KPIs, line plots, bar charts, scatter charts, and summary tables across modules
Research and report quality	Security-cost reasoning, difficulty-period interpretation, and modular documentation

The project also benefits from its modular organization. Each feature is isolated in its own file, which makes the repository easier to understand and gives the Git history a natural progression. This is an important practical aspect of the work because the assignment values progressive development rather than a single last-minute code dump.

11. Limitations and Future Improvements

Even in its final form, the project still has room for improvement. Some analytical models in the optional modules are intentionally simplified in order to keep the implementation explainable and robust. The security-risk plot in M6 is illustrative rather than a full implementation of Nakamoto's exact probabilistic model, and the anomaly-detection modules in M4 and M7 serve as strong baselines rather than advanced trained machine-learning systems.

A future version could extend the AI side by introducing supervised prediction or a more advanced unsupervised model trained on longer historical windows. Another interesting direction would be a more detailed fee-estimation component based on mempool conditions, or a second blockchain for comparative study. On the cryptographic side, M5 could be expanded with richer transaction context and graphical tree rendering.

However, these limitations do not undermine the current project. In fact, they reinforce one of its strengths: the implementation remains understandable, explainable, and directly tied to the concepts taught in class. The dashboard favors correctness and clarity over unnecessary complexity.

12. Conclusion

The CryptoChain Analyzer Dashboard achieves the central objective of the assignment: it transforms live Bitcoin blockchain data into a coherent, technically grounded, and professionally presented dashboard. The final version integrates applied cryptography, real-time monitoring, blockchain structure verification, security reasoning, and AI-based analysis within a single modular interface.

Its strongest contributions are the direct cryptographic checks in M2 and M5, the meaningful protocol-level analysis in M3 and M6, and the comparative AI perspective offered by M4 and M7. Together, these modules show not only that the student can write code that runs, but also that he can connect theory, data, and software engineering into a complete analytical product.

For these reasons, the project should be considered a strong and ambitious final submission: it covers the required components, implements the optional extensions, and demonstrates a serious effort to go beyond a minimal dashboard into a full educational and technical tool.

13. References

- Bitcoin Whitepaper: Satoshi Nakamoto, Bitcoin: A Peer-to-Peer Electronic Cash System (2008).
- Mempool.space REST API documentation, used for live Bitcoin block, transaction, and Merkle proof data.
- Python hashlib documentation, used for local SHA-256 and double SHA-256 verification.
- Streamlit documentation, used for the dashboard interface and interactive components.